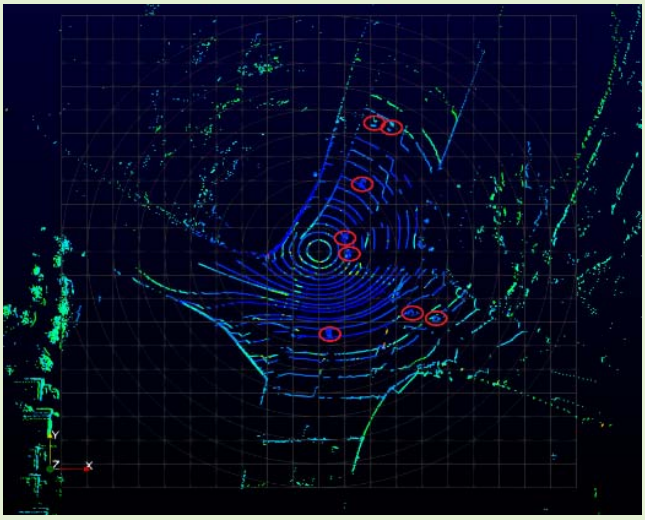
An Unsupervised Clustering Method for Processing Roadside LiDAR Data With Improved Computational Efficiency

Yibin Zhang, Nischal Bhattarai, Junxuan Zhao, Hongchao Liu, and Hao Xu

***Abstract*—In transportation, LiDAR has been primarily used in autonomous vehicles to assist self-driving until recently when people realized it could also be installed at the roadside to support connected vehicles and infrastructure systems. Unlike onboard LiDAR sensors used in autonomous vehicles, roadside applications must perform complete background filtering and clustering as well as tracking real-time traffic movements within the detection zone. This paper presents an unsupervised clustering method for roadside or infrastructure-based LiDAR applications. It first converts 3D LiDAR data points into 2D so that only target points (after background filtering) will be saved in the channel-azimuth 2D structure; then, a method combining the region growing algorithm and counted component labeling is used to perform clustering. Lastly, a merging process is conducted to enhance the connected component labeling method for better outcomes. Experimental studies demonstrate that the proposed method could reach 0.011s per frame (10 Hz sensor rotation frequency) in clustering while maintaining high accuracy.**



***Index Terms*—Connected component labeling, computational efficiency, object clustering, region growing, roadside LiDAR.**

I. Introduction

AUTOMATED driving systems are revolutionary technologies that could shape our future by minimizing and eventually eliminating human errors in driving. However, autonomous vehicles have some critical barriers to overcome towards full self-driving automation. The obstacles may include, but are not limited to, the challenge to identify objects, the limits of computation power of onboard computers, and the difficulty to react appropriately to any scenario a car may encounter while on the road. A solution to the problem is to develop infrastructure-based systems that can help self-driving cars better understand roads and help nonmotorized road users get active protection when an automatic system fails. Studies on the application of LiDAR technology at the infrastructure side and the development of connected vehicle and infrastructure systems were recently conducted by several groups of researchers [1]–[6]. By timely collecting and analyzing data from the roadside LiDAR and sending dedicated information to cars (e.g., broadcasting traffic light's color directly to a vehicle's computer), researchers expect the infrastructure-based system can take the substantial burden out of an autonomous vehicle's onboard computers [7] and significantly improve the efficiency of self-driving.

Infrastructure-based LiDAR usually works independently and thus has a heavier computation load. A busy intersection can easily have thousands of vehicles during peak hours along with other modes of road users such as pedestrians and cyclists. Roadside or infrastructure-based LiDAR sensors need to monitor every road user and track their movements in real-time to capture risky behaviors (such as red-light running) and trigger protective actions. Another challenge to processing

Manuscript received February 26, 2022; accepted April 8, 2022. Date of publication April 13, 2022; date of current version May 31, 2022. The associate editor coordinating the review of this article and approving it for publication was Prof. Piotr J. Samczynski. *(Corresponding author: Hongchao Liu.)*

Yibin Zhang, Nischal Bhattarai, Junxuan Zhao, and Hongchao Liu are with the Department of Civil, Environmental, and Construction Engineering, Texas Tech University, Lubbock, TX 79409 USA (e-mail: yibin.zhang@ttu.edu; nischal.bhattarai@ttu.edu; junxuan.zhao@ttu.edu; hongchao.liu@ttu.edu).

Hao Xu is with the Department of Civil and Environmental Engineering, University of Nevada, Reno, NV 89557 USA (e-mail: haox@unr.edu).

Digital Object Identifier 10.1109/JSEN.2022.3166957

roadside LiDAR data applications is its cost. Commercially available LiDAR sensors include products with 1, 4, 8, 16, 32, 64, and 128 laser channels, and the number of laser channels determines the accuracy and detection range as well as the cost. Automated vehicles are primarily equipped with 128-channel products, while roadside applications must use low-cost products with fewer channels, limited range, and lower resolution [6], [8]. The detection accuracy may also be affected by other factors, such as retrieved aerosol [9]. Despite the limits of using low-cost LiDAR sensors, an infrastructure-based system must be able to detect and analyze the situation, activate the warning scheme, and deliver the information to pedestrians, bicyclists, and drivers in a very short time horizon. To this end, current methods for point cloud data processing, esp. the machine learning methods for background filtering, object identification [10], and movement tracking, need to be thoroughly reviewed and further developed for infrastructure-based applications; copy of existing methods for autonomous vehicles will not work. For instance, for onboard LiDAR, the random point cloud should be removed [11], whereas, for roadside LiDAR, the point cloud of static objects like buildings and trees should be removed.

II. Related Works

Background filtering, object clustering, object classification, and object tracking are four major steps for processing roadside LiDAR data; this study is focused on clustering. Both supervised classification [12], [13] and unsupervised clustering [14] are used in object grouping. In supervised classification, the class or label of an object must be given; machine learning methods such as neural networks [15] and decision trees [16] are commonly used in supervised clustering. To automatically cluster objects in real-time, an unsupervised recognition method should be applied because it is impossible to specify the labels beforehand. Density-based spatial clustering of applications with noise (DBSCAN), K-means clustering methods, and their variations are popular unsupervised methods [17], [18]. However, K-means requires the number of clusters as an input [19], which does not fit with infrastructure-based LiDAR applications in which the number of road users is unknown. DBSCAN algorithm is better suited for vehicle clustering as it separates clusters based on the density of points in the point cloud without requiring the number of clusters as a hyperparameter. Usually, DBSCAN can produce fairly good results if computation time is not an issue, making it good for offline applications.

Over-segmentation (see Figure 1) is also a problem that decreases the precision of clustering [20]. The Figure on the top of Figure 1 shows the raw data of vehicles after the background filtering. The circled object on the top of Figure 1 should be one vehicle. The Figure on the bottom is the presentation of over-segmentation. X-axis and Y-axis represent the horizontal and vertical distances from the top-view, and the location of LiDAR is represented by the star at coordinates (0,0). As can be seen, one vehicle was identified as two objects (vehicle #1 and vehicle #2) if the over-segmentation is not stressed. Researchers used the Gaussian Process (GP) regression [21] to improve clustering results. However, the fact

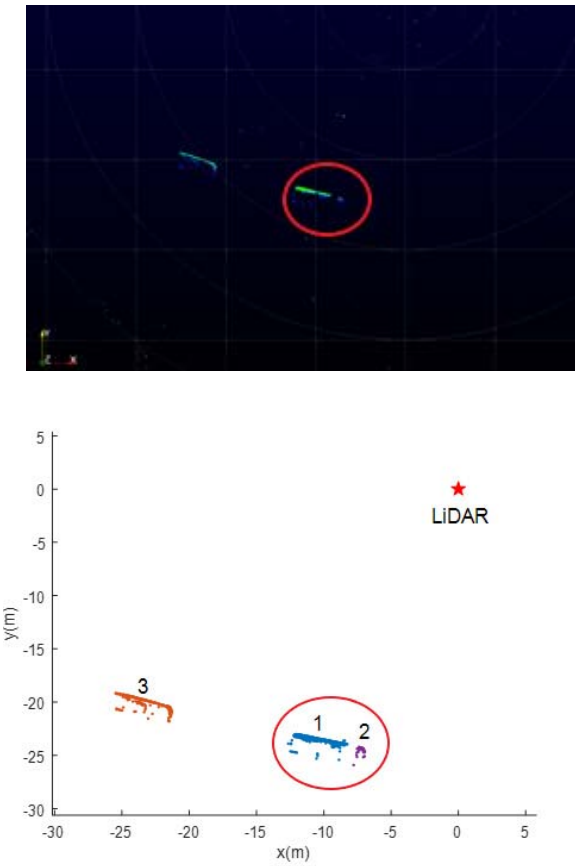


Fig. 1. Over-segmentation of vehicle.

that the GP regression is applied to every point only works for the situation containing extremely large or small objects due to time consideration. This may not be suitable for infrastructure-based LiDAR sensors because the two similar size objects may occur due to occlusion [22].

The introduction by far has illustrated the differences between the applications of vehicle-mounted and infrastructure-based LiDAR sensors as well as the major challenges to processing infrastructure-based LiDAR data in real-time. This paper presents a so-called counted region growing method derived from the common concept of region growing [23], [24] and connected component labeling [25], [26] approach to improve the computation efficiency of clustering. A merging process is added after applying the counted region growing to overcome the problem of over-segmentation.

The rest of this paper is structured as follows: section III introduces the 2D data structure; section IV presents the counted region growing method and the merging process; section V presents the experimental study along with discussions, and section VI concludes the study with the cons and pros of the proposed approach.

III. 2D Data Structure

Converting 3D data structure to 2D [27], [28] is a common procedure for point cloud data processing, which can be done by established algorithms such as SLAM (Simultaneous Localization and Mapping) [29], [30]. As aforementioned, LiDAR products use a different number of paired lasers to

measure the distance to objects. Lasers are fired from top to bottom while the LiDAR sensor rotates around the center to form a circle. Thus every data point can be located by a vertical angle (ω), azimuth (α), and R (distance between LiDAR and the data point.

More specifically, the sensor uses the time-of-flight (ToF) [31] methodology to store each laser pulse's fire time and direction. After the laser hits an obstacle and is reflected, LiDAR registers the time-of-acquisition. The rotation rate influences the resolution of horizontal angle ($\alpha\alpha$), which can be obtained from the following equation:

$$\alpha_{resolution} = \frac{s}{60} \times 360 \times t \tag{1}$$

where s is the rotation speed (cycle/min), t is the time of firing sequence (s/cycle).

The 2D data structure can be made as follows:

$$grid\,(x, y) = R \tag{2}$$

where,

$$x = (\alpha + offset)/\alpha_{resolution} \tag{3}$$

$$y = n \ when \ \omega \ is \ \ fired \ by \ nth \ laser \tag{4}$$

It is notable that since laser emits in pairs, the precise horizontal angle is not the exact value of α; instead, an offset is needed to amend the value of α. Due to the effect from the offset, the value of $\alpha + offset$ may exceed the range from [0, 360], therefore, to obtain the accurate reference, it can be converted to a different scale by covering all values using the angular resolution as:

If $\alpha + offset > 360$:

$$x = (\alpha + offset - 360)/\alpha_{resolution} \tag{5}$$

If $\alpha + offset < 360$:

$$x = (\alpha + offset + 360)/\alpha_{resolution} \tag{6}$$

A LiDAR that contains 32 beam lasers is used in this study. All 32 laser beams are fired in pairs and recharged periodically at an interval called Firing Sequence, and Firing Sequence determines the vertical angle of data points. Meanwhile, the sensor's motor is set to 10 Hz, so the value of $\alpha_{resolution}$ is obtained to be 0.2 since the time for each fire sequence is 55.296μs.

Based on the analysis above, a 2D data structure was obtained to store the data points. The column ranges from [1, 32] because there are 32 lasers, and the row ranges from [1, 1800] because the sensor generates one scan (360-degree rotation) with 1800 azimuth intervals ($\alpha_{resolution}$ = 0.2-degree), R is the 3D distance between the sensor and each data point (if exists). The offsets are provided as unique values for a specific type of LiDAR sensor from manufacturers. As shown in Table I, the numbers in the first column represent the azimuth intervals (x), the numbers in the first row represent the laser order (y), and the numbers in the blocks are R values (3D distance). 0 means no data in that block. No data means that the laser fired by LiDAR does not hit any objects or data is filtered after the background filter.

TABLE I
A SAMPLE OF THE 2D DATA STRUCTURE

x \ y / R	…	13	14	15	…
…	…	…	…	…	…
597	0	0	10.80	0	0
598	0	10.83	10.88	10.77	0
599	0	10.84	10.64	10.71	0
600	0	10.85	10.65	10.73	0
601	0	10.76	10.66	10.74	0
…	…	…	…	…	…

Note: x: azimuth intervals. y: laser order. R:D distance. The meanings of the parameters are the same for the following Table.

IV. METHODOLOGICAL APPROACH

A. Counted Region Growing

Based on the 2D data structure, a revised region growing method dubbed "counted region growing" was developed to automatically cluster vehicles without prior knowledge of the number of vehicles. The counted region growing is featured by a combined region growing and connected component labeling, introduced in this section.

As the 2D structure is like Image data, each block can be treated as a pixel. Similar to region growing, two clusters are determined by the distance between the seed point and the neighbors. The criteria between two sets of clusters A and B are as follows [32]:

$$min\{d\,(a, b) : a \in A, b \in B\} \tag{7}$$

where $d\,(a, b)$ is the distance between instances a and b that belong to clusters A and B, respectively.

Then, clustering can be performed according to the following steps:

1) *Find the seed point: set the seed point by searching the unlabeled blocks from the first to the last grid block until it finds one with a nonzero value.*
2) *Grow the region of seed point: search the neighbors of the seed point. If the difference between the values of the neighbors and seed point satisfies a given condition, set the neighbor as the new seed point and search again until none of the neighbors satisfies the condition.*
3) *Label the region: label each block in the same region and go back to step one, continue searching from the previous seed point.*
4) *Delete the unnecessary regions: after the search is completed for the entire grid, delete the regions whose number of data points is less than a threshold.*

It is notable that the location of any value in a matrix can be presented in either linear indexing or an index with two subscripts. For example, for a matrix A with 3 rows and 3 columns, *A(4)* is the same as *A(1,2)* since the columns are counted firstly. Detailed steps of the counted region growing method can be summarized as followed on pseudocode.

Since the detective zone of LiDAR is a circle or a semi-sphere that is continuous, once the searching area of seed

TABLE II
A SAMPLE OF DISCONNECTED DATA

x \ y R	…	13	14	15	…
…	…	…	…	…	…
597	0	0	10.80	10.88	0
598	0	0	0	10.77	0
599	0	10.84	0	0	0
600	0	10.85	10.65	0	0
601	0	10.76	10.66	0	0
…	…	…	…	…	..

TABLE III
A SAMPLE OF ADJACENT DATA OF TWO-VEHICLES

x \ y R	…	13	14	15	…
…	…	…	…	…	…
597	0	0	22.80	22.65	22.59
598	0	10.83	22.88	22.70	22.81
599	0	10.84	22.64	22.73	10.77
600	0	10.85	10.65	10.73	10.66
601	10.79	10.76	10.66	10.74	10.81
…	…	…	…	…	…

point exceeds the row limitation - for instance, the searching area of *A(1799,13)* will exceed 1800, which is the boundary of the 2D grid, it should contain the rows that go back from the first row, vice versa. The reason why $GRID(X_i)$ is set to -1 after confirming that $grid(X_i)$ is 0 is to mark X_i as a block that has been searched to decrease computation time.

Ideally, the data points of a vehicle collected by a LiDAR sensor should be similar in value and close to each other in the 2D grid, as shown in Table I. In reality, however, some objects of certain materials may not reflect laser beams but rather absorb them, causing the collected distance value to be disconnected. Table II shows that the upper triangle and lower triangle should be one vehicle, however, the data are disconnected. Another example shown in TABLE III is that the data points collected from two vehicles are connected. In reality, they are two different vehicles (each color represents one single vehicle.)

Selection of the number of neighbors (n) for searching and the distance threshold (d) is critical to encounter the above two problems. However, since the traffic flow varies, such as different headspaces or lane widths, and installations, different types of LiDAR or occlusion situations, the determination of n and d should be based on real conditions. Especially for n, the selection of n needs to consider the type of LiDAR sensor and the tradeoff between the searching area and the noise.

The type of LiDAR sensor, and the actual field environment also affect the selection. As for d, a number less than 1.5 m (the general width of a vehicle) should be considered because larger values will increase the error of two adjacent vehicles being identified as one.

TABLE IV
A SAMPLE OF UNCONNECTED DATA OF ONE-VEHICLES

x \ y R	12	13	14	15	16
586	10.91	10.75	10.73	10.67	0
587	0	10.84	10.80	10.88	0
588	0	0	0	0	0
…	…	…	…	…	…
599	0	0	0	0	0
600	0	10.76	10.66	10.81	0
601	10.78	10.90	10.82	10.76	10.83

B. Merging Process

Careful selection of the parameters is important but not enough to offset the errors caused by occlusion and other real-world situations such as the gap between a truck tractor and a semi-trailer; as shown in Table IV, in some cases, the detached blocks may be too large.

This section introduces a merging process to merge the disconnected data points when they belong to one object. In order not to compromise computational efficiency, the strategy of the proposed merging process is to only compare the nearest blocks of each cluster after the counted region growing. If the difference of the values of the nearest blocks is within the threshold, these clusters are regarded as one cluster.

To ensure the separated clusters, namely A_1 and A_2, are from a single vehicle and to find the nearest blocks of the separated clusters, three conditions need to be examined for $A_1(x_{n1}, y_{n1})$ and $A2\ (x_{n2}, y_{n2})$:

The separated clusters have the common columns in the 2D grid. The common columns indicate the objects are fired by the same laser ID. The equation could be represented as follows.

$$y_n = y_{n1} \cap y_{n2} \neq \emptyset \tag{8}$$

The row intervals (the difference between rows) of the common columns are the nearest and within the threshold. Finding the nearest row interval improves the speed of the merging process, while the threshold guarantees that the clusters are close to each other. In detail:

$$|x_1 - x_2| = min|x'_{n1}x'_{n2}| \tag{9}$$

And

$$|x_1 - x_2| < T_r \tag{10}$$

where x'_{n1} and x'_{n2} are the largest row numbers or smallest row numbers corresponding to the common columns y_n for clusters A_1 and A_2, respectively. T_r is the threshold for row intervals.

Find y, i.e., the column where x_1 and x_2 exist.

The difference of the values in the nearest blocks is within the threshold to verify if they belong to the same vehicle.

$$|A_1(x_1, y) - A_2(x_2, y)| < T_d \tag{11}$$

where T_d is the threshold for the difference of the values in the nearest blocks. For clusters that do not have common columns,

Initialize **Grid** to 0 with the same size as **grid**
Input the **distance_threshold** and **number_threshold**
Set **indicator** to 1
Initialize the **neighborhood_list**
While **block_counter** is less than the total_block of grid
  If **grid(block_counter)** is equal to 0
    Set **Grid(block_counter)** to -1
    Go to next iteration
  End
  If **Grid(block_counter)** is not euqal to 0
    Go to next iteration
  End
  Set **nerghbor_index** of **block_counter** by calling the function of **Getneighbor**
  (**Getneighbor** finds out the nerghbor's positions)
  Set **Grid(block_counter)** to **indicator**
  Set **Unsearched_block** to the positions where **Grid(nerghbor_index)** equal to 0
  If |**grid(Unsearched_block)** - **gird(block_counter)**| (difference) is less than **distance_threshold**
    Set **Seed** to the positions where the difference is less than **distance_threshold**
    Set **List_length** to the number of **Seed**
    Set **neighborhood_list(1:List_length)** to **Seed**
    Set **Grid(Seed)** to **indicator**
    Set **Seed_counter** to 1
    While **neighborhood_list** has non zero values
      ReSet **nerghbor_index** of **neighborhood_list(Seed_counter)**
      ReSet **Unsearched_block** to the positions where **Grid(nerghbor_index)** equal to 0
      If |**grid(Unsearched_block)** - **grid(neighborhood_list(Seed_counter))**| (difference) is less than **distance_threshold**
        ReSet **Seed** to the positions where the difference is less than **distance_threshold**
        Set **List_length_More** to the number of **Seed**
        Set **neighborhood_list(List_length:List_length_More)** to **Seed**
        Set **Grid(Seed)** to **indicator**
        Set **List_length** to **List_length** + **List_length_More**
        Set **neighborhood_list(Seed_counter)** to 0
        Add 1 to **Seed_counter**
      Else
        Set **neighborhood_list(Seed_counter)** to 0
        Add 1 to **Seed_counter**
      End
    End
    Add 1 to **indicator**
  Else
    Add 1 to **indicator**
    Go to next iteration
  End
End
Set **Grid(Grid==-1)** to 0
Set **Count_indicator** to the number of blocks that each indicator occupants
Set **Todelete_indicator** to the **indicators** whose **Count_indicator** is less than **number_threshold**
Set **Grid(Grid==Todelete_indicator)** to 0
Return **Grid**

once the difference of the row numbers of their nearest blocks is within the T_r, the criterion of judging whether two clusters belong to one object is to compare the minimum and maximum value of each cluster. Again, once the difference of value either for minimum or maximum is within T_d, it is believed that these two clusters are one object.

V. Experimental Study

The data used in this study was collected by VLP-32 of Velodyne LiDAR sensors at the Veterans & Mira Loma intersection (Figure.2) at the City of Reno, Nevada. The portable LiDAR was equiped at the height [8] of 2 meters. The speed limit for Veterans road segment is 55 MPH while

Fig. 2. Veterans & mira loma site.

TABLE V
SEARCH AREA OF THE SEED POINT

x \ y	…	13	14	15	…
…	…	…	…	…	…
597	0	0	10.80	0	0
598	0	10.83	10.88	10.77	0
599	0	10.84	10.64	10.71	0
600	0	10.85	10.65	10.73	0
601	0	10.76	10.66	10.74	0

TABLE VI
MERGING RESULT OF UNCONNECTED DATA OF ONE-VEHICLES

x \ y	12	13	14	15	16
586	10.91	10.75	10.73	10.67	0
587	0	10.84	10.80	10.88	0
588	0	0	0	0	0
…	…	…	…	…	…
599	0	0	0	0	0
600	0	10.76	10.66	10.81	0
601	10.78	10.90	10.82	10.76	10.83

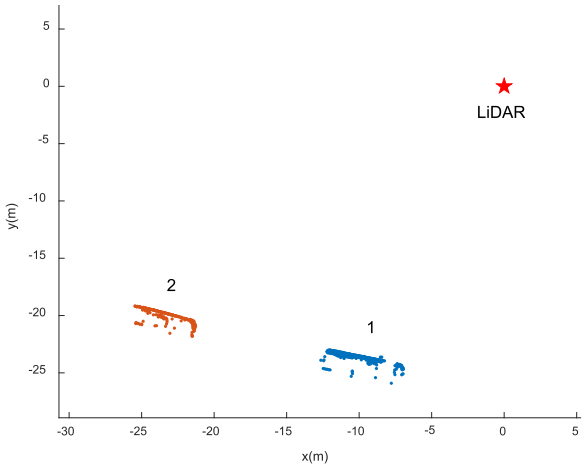


Fig. 3. The result of the merging process.

for Mira Loma road segment is 30 MPH. According to a study performed at a similar site, Veterans Pkwy & E Greg St intersection, average speed was about 15 MPH at the stop bar for northbound vehicles that slowed but did not stop.

Before converting 3-D data to the 2-D structure, background filtering was applied to filter the background data points from all frames, and the method was introduced in a previous study of the authors [6]. The algorithm was programmed in Matlab and run on a laptop with Intel i7 CPU @2.6 GHz and 16 GB RAM. The total clustering time for 1000 frames was 10.644302s, which is almost 6 times faster than DBSCAN's 60.178293s.

In this study, 24 neighbors of a seed point are searched. As shown in Table V, the seed point highlighted in red is centered, while the gray blocks represent the search area of the seed point. 10 data points are the minimum requirement for a cluster, which means an object should contain at least 10 data points; otherwise, the object will be deleted.

Table VI and Figure 3 are two ways to demonstrate the result of the merging process. From the data view, Table VI shows that the unconnected data of one vehicle in Table V is identified as one object. Figure 3 visualizes the results of the merging process. The two clusters (vehicle #1 and vehicle #2) in Figure 1 are merged into a single vehicle (vehicle #1).

Constrained by the misalignment of LiDAR [33] and other conditions relevant to field implementation, such as roadway geometry and traffic conditions, there should be no universal threshold values that apply to all environments. It is found that based on the conditions our dataset was built, the method reaches the best performance when $d = 1$, $T_r = 20$, $T_d = 0.5$ (see Table VII). In Table VII, the numbers of vehicles identified by the proposed method with different parameters, namely the distance of counted region growing (d), the distance of merging process (T_d), the threshold for row intervals (T_r), are given. The results are compared to the number of vehicles in the ground truth. The ground truthground truth was obtained by manually counting the vehicles through Velodyne LiDAR visualization software (Veloview) using original .pcap file, which is 2170 vehicles in 550 frames. The rate here is just the total number of detected vehicles divided by the ground truth. The number of vehicles identified by the proposed method is close to the true number of vehicles since all rates are above 95%.

However, since the situation in which vehicles are divided into two objects due to large occlusion exists, the number of vehicles being identified correctly should be paid more attention. To further demonstrate the accuracy of the proposed method ($d = 1$, $T_r = 20$, $T_d = 0.5$), the numbers of vehicles being identified correctly are compared to the ground-truth numbers of vehicles in Table VIII.

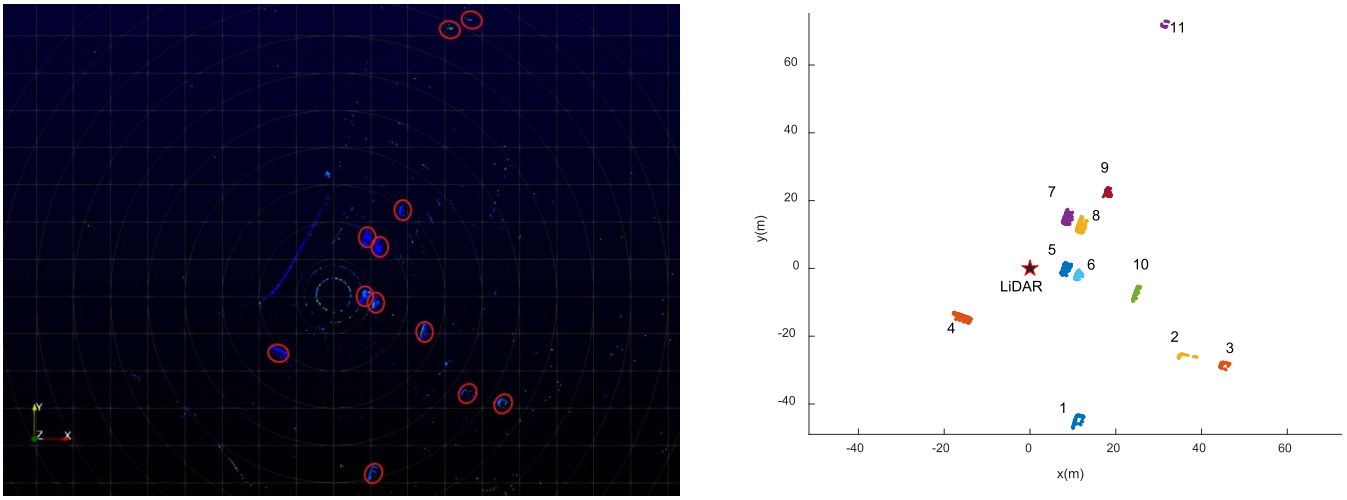
The accuracy in Table VIII is the ratio of the number of vehicles being identified correctly and the ground truth. The mean accuracy is 0.9668, as shown in Table VIII.

As stated in [6] although the detection range of LiDAR could reach 100m for a 16-laser LiDAR, the effective range is only 30m for high-quality detection.

TABLE VII
NUMBER OF VEHICLES IDENTIFIED BY DIFFERENT PARAMETERS

	0-149 Frames	150-299 Frames	300-449 Frames	449-549 Frames	Total	Rate
d=1, T_r=20, T_d = 0.5	412	258	629	834	2133	0.9829
d=3, T_r=20, T_d = 0.5	416	256	647	813	2132	0.9824
d=1.5, T_r=20, T_d = 2.5	414	257	622	800	2093	0.9645
d=1.5, T_r=50, T_d = 0.5	415	258	626	829	2128	0.9806
d=2, T_r=30, T_d = 1.5	415	252	633	825	2125	0.9792
Ground Truth	414	256	647	853	2170	/

Note: d: distance of counted region growing. T_d: distance of merging process. T_r: threshold for row intervals



A. 12 vehicles in ground truth B. 11 vehicles are detected

Fig. 4. Clustering result of other frames.

TABLE VIII
ACCURACY RESULT

Frames	Ground Truth	Correctly Identified	Accuracy
0-49	158	158	1.0000
50-99	144	143	0.9930
100-149	112	110	0.9821
150-199	69	69	1.0000
200-249	85	84	0.9882
250-299	102	97	0.9510
300-349	125	114	0.912
350-399	249	239	0.9598
400-449	273	266	0.9743
450-499	338	321	0.9497
500-549	515	497	0.9650
Total	2170	2098	0.9668

In the case of 32-laser LiDAR sensors that are used in this study, 50-75m seems to be the upper limit for best detection quality, albeit the 200m range claimed by the manufacturer. Occlusion and the point density would be two major factors that influence the accuracy. For example, the error for 500-549 frames is mostly from distant vehicles that have too few points to identify, even for manually counting. As shown in Figure 4, the furthest vehicle is hard to recognize, but this vehicle could be identified by the forward frames. The proposed method detects 11 vehicles since the furthest vehicle only has a few points. However, Figure 4 (A) and (B) shows a case with a vehicle about 60 meters away from the sensor, which was clearly identified (vehicle #11).

VI. CONCLUSION

An unsupervised clustering method for roadside LiDAR application was presented. The proposed method is developed based on the region growing algorithm coupled with counted component labeling and a revised merging process. A major thrust of the study lies in its capability to improve computation speed and over-segmentation, which is critical to roadside LiDAR application. At a speed of 0.011s per frame, the algorithm lays a solid ground for real-world applications such as red-light running and jaywalking protections. The threshold values were determined by trials, which should be used as references rather than recommendations considering the variety of real-world situations. Studies that include nonmotorized road users such as pedestrians and cyclists are currently ongoing.

REFERENCES

[1] S. Gargoum and K. El-Basyouny, "Automated extraction of road features using LiDAR data: A review of LiDAR applications in transportation," in *Proc. 4th Int. Conf. Transp. Inf. Saf. (ICTIS)*, Aug. 2017, pp. 563–574.

[2] A. Lesani, E. Nateghinia, and L. F. Miranda-Moreno, "Development and evaluation of a real-time pedestrian counting system for high-volume conditions based on 2D LiDAR," *Transp. Res. C, Emerg. Technol.*, vol. 114, pp. 20–35, May 2020.
[3] J.-S. Lee, J.-H. Jo, and T.-H. Park, "Segmentation of vehicles and roads by a low-channel lidar," *IEEE Trans. Intell. Transp. Syst.*, vol. 20, no. 11, pp. 4251–4256, Nov. 2019.
[4] M. Lato, J. Hutchinson, M. Diederichs, D. Ball, and R. Harrap, "Engineering monitoring of rockfall hazards along transportation corridors: Using mobile terrestrial LiDAR," *Natural Hazards Earth Syst. Sci.*, vol. 9, no. 3, pp. 935–946, Jun. 2009.
[5] A. Klautau, N. Gonzalez-Prelcic, and R. W. Heath, Jr., "LIDAR data for deep learning-based mmWave beam-selection," *IEEE Wireless Commun. Lett.*, vol. 8, no. 3, pp. 909–912, Jun. 2019.
[6] J. Zhao, H. Xu, H. Liu, J. Wu, Y. Zheng, and D. Wu, "Detection and tracking of pedestrians and vehicles using roadside LiDAR sensors," *Transp. Res. C, Emerg. Technol.*, vol. 100, pp. 68–87, Mar. 2019.
[7] H. Gao, B. Cheng, J. Wang, K. Li, J. Zhao, and D. Li, "Object classification using CNN-based fusion of vision and LIDAR in autonomous vehicle environment," *IEEE Trans. Ind. Informat.*, vol. 14, no. 9, pp. 4224–4231, Sep. 2018.
[8] J. Zhao, H. Xu, Y. Tian, and H. Liu, "Towards application of light detection and ranging sensor to traffic detection: An investigation of its built-in features and installation techniques," *J. Intell. Transp. Syst.*, vol. 26, no. 2, pp. 213–234, Mar. 2022.
[9] A. Lay-Ekuakille, F. De Tomasi, M. R. Perrone, and A. Trotta, "Output characterization of ground-based and integrated optical sensor for retrieved aerosol error minimization," in *Proc. 20th IEEE Instrum. Technol. Conf.*, vol. 2, May 2003, pp. 1018–1021.
[10] G. Nash and V. Devrelis, "Flash LiDAR imaging and classification of vehicles," in *Proc. IEEE Sensors*, Oct. 2020, pp. 1–4.
[11] M. A. Yahya, S. Abdul-Rahman, and S. Mutalib, "Object detection for autonomous vehicle with LiDAR using deep learning," in *Proc. IEEE 10th Int. Conf. Syst. Eng. Technol. (ICSET)*, Nov. 2020, pp. 207–212.
[12] Y. Zhang, Y. Yin, D. Guo, X. Yu, and L. Xiao, "Cross-validation based weights and structure determination of Chebyshev-polynomial neural networks for pattern classification," *Pattern Recognit.*, vol. 47, no. 10, pp. 3414–3428, 2014.
[13] G. Ou and Y. L. Murphey, "Multi-class pattern classification using neural networks," *Pattern Recognit.*, vol. 40, no. 1, pp. 4–18, Jan. 2007.
[14] C. Zhong, D. Miao, and R. Wang, "A graph-theoretical clustering method based on two rounds of minimum spanning trees," *Pattern Recognit.*, vol. 43, no. 3, pp. 752–766, 2010.
[15] V. M. Lidkea, R. Muresan, and A. Al-Dweik, "Convolutional neural network framework for encrypted image classification in cloud-based ITS," *IEEE Open J. Intell. Transp. Syst.*, vol. 1, pp. 35–50, 2020.
[16] T. Champahom, S. Jomnonkwao, V. Chatpattananan, A. Karoonsoontawong, and V. Ratanavaraha, "Analysis of rear-end crash on Thai highway: Decision tree approach," *J. Adv. Transp.*, vol. 2019, pp. 1–13, Nov. 2019.
[17] F. Morsdorf, E. Meier, B. Kötz, K. I. Itten, M. Dobbertin, and B. Allgöwer, "LIDAR-based geometric reconstruction of boreal type forest stands at single tree level for forest and wildland fire management," *Remote Sens. Environ.*, vol. 92, no. 3, pp. 353–362, 2004.
[18] M. Tonini and A. Abellan, "Rockfall detection from terrestrial LiDAR point clouds: A clustering approach using R," *J. Spatial Inf. Sci.*, vol. 8, no. 1, pp. 95–110, Jun. 2014.
[19] D. T. Pham, S. S. Dimov, and C. D. Nguyen, "Selection of K in K-means clustering," *Proc. Inst. Mech. Eng., C, J. Mech. Eng. Sci.*, vol. 219, no. 1, pp. 103–119, 2005.
[20] M.-O. Shin, G.-M. Oh, S.-W. Kim, and S.-W. Seo, "Real-time and accurate segmentation of 3-D point clouds based on Gaussian process regression," *IEEE Trans. Intell. Transp. Syst.*, vol. 18, no. 12, pp. 3363–3377, Dec. 2017.
[21] C. K. Williams and C. E. Rasmussen, *Gaussian Processes for Machine Learning*, vol. 2, Cambridge, MA, USA: MIT Press, no. 3, 2006, p. 4.
[22] X. Song *et al.*, "Augmented multiple vehicles' trajectories extraction under occlusions with roadside LiDAR data," *IEEE Sensors J.*, vol. 21, no. 19, pp. 21921–21930, Oct. 2021.
[23] D. H. Ballard and C. Brown, *Computer Vision*. Berlin, Germany: Prentice hall, 1982.
[24] S. A. Hojjatoleslami and J. Kittler, "Region growing: A new approach," *IEEE Trans. Image Process.*, vol. 7, no. 7, pp. 1079–1084, Jul. 1998.
[25] L. He, Y. Chao, K. Suzuki, and K. Wu, "Fast connected-component labeling," *Pattern Recognit.*, vol. 42, no. 9, pp. 1977–1987, 2009.
[26] M. B. Dillencourt, H. Samet, and M. Tamminen, "A general approach to connected-component labeling for arbitrary image representations," *J. ACM*, vol. 39, no. 2, pp. 253–280, Apr. 1992.
[27] H. Wang, B. Wang, B. Liu, X. Meng, and G. Yang, "Pedestrian recognition and tracking using 3D LiDAR for autonomous vehicle," *Robot. Auto. Syst.*, vol. 88, pp. 71–78, Feb. 2017.
[28] B. Wu, A. Wan, X. Yue, and K. Keutzer, "SqueezeSeg: Convolutional neural nets with recurrent CRF for real-time road-object segmentation from 3D LiDAR point cloud," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, May 2018, pp. 1887–1893.
[29] T. Bailey and J. Nieto, "Scan-SLAM: Recursive mapping and localisation with arbitrary-shaped landmarks," in *Proc. Robot., Sci. Syst. Conf. (RSS)*, 2008, pp. 1–12.
[30] S. Huang and G. Dissanayake, "Convergence and consistency analysis for extended Kalman filter based SLAM," *IEEE Trans. Robot.*, vol. 23, no. 5, pp. 1036–1049, Oct. 2007.
[31] G. Chen, C. Wiede, and R. Kokozinski, "Data processing approaches on SPAD-based d-TOF LiDAR systems: A review," *IEEE Sensors J.*, vol. 21, no. 5, pp. 5656–5667, Mar. 2021.
[32] A. Saxena *et al.*, "A review of clustering techniques and developments," *Neurocomputing*, vol. 267, pp. 664–681, Dec. 2017.
[33] S. Oh, J.-H. You, A. Eskandarian, and Y.-K. Kim, "Accurate alignment inspection system for low-resolution automotive LiDAR," *IEEE Sensors J.*, vol. 21, no. 10, pp. 11961–11968, May 2021.

Yibin Zhang is pursuing the Ph.D. degree with the Department of Civil, Environmental, and Construction Engineering, Texas Tech University. His research interests include GIS in transportation, highway safety, and connected-vehicle applications with roadside LiDAR sensors.

Nischal Bhattarai is pursuing the Ph.D. degree with the Department of Civil, Environmental, and Construction Engineering, Texas Tech University. His research interests include traffic design and operation, traffic impact analysis, traffic safety, and GIS in transportation.

Junxuan Zhao received the B.E. degree in electrical engineering and automation from Anhui Agricultural University, China, in 2013, the M.S. degree in electrical engineering from Colorado State University in 2015, and the Ph.D. degree from Texas Tech University in 2019. Her research interests include connected-vehicle applications with roadside LiDAR sensors, intelligent transportation systems, GIS in transportation, and GNSS.

Hongchao Liu received the Ph.D. degree in transportation engineering from the University of Tokyo and the M.S.C.E. degree in transportation planning from Tsinghua University. He is a Professor with the Department of Civil, Environmental and Construction Engineering, Texas Tech University. His research interests include traffic operation and control, intelligent transportation systems, traffic flow theory, and traffic simulation.

Hao Xu received the B.E. and M.E. degrees in automation from the University of Science and Technology of China in 2007 and 2004, respectively, and the Ph.D. degree from Texas Tech University in 2011. He is an Associate Professor with the Department of Civil and Environmental Engineering, University of Nevada, Reno. His research areas include connected-vehicle applications with roadside LiDAR sensors, highway safety, and intelligent transportation systems.